

MERN Real-Time Chat App

Frontend System Design + AI Coding Prompt

Purpose: Build a production-ready React frontend for a simple one-to-one real-time chatting web application. The frontend must integrate cleanly with the existing Node.js + Express + MongoDB + JWT + Socket.IO backend described in the companion backend system design.

1. Frontend Technology Stack

Layer	Technology	Purpose
UI	React.js	Component-based chat application interface
Build	Vite	Fast local development and production build
Styling	Tailwind CSS	Responsive UI, states, layout and theming
Routing	React Router	Authentication and protected application routes
HTTP	Axios or Fetch	REST API communication
Real-time	Socket.IO Client	Messages, presence, typing and read receipts
State	Context API or Zustand	Auth, user, chat and real-time UI state
Forms	React Hook Form + Zod	Client-side validation and form handling
Icons	Lucide React	Consistent interface icons

2. Frontend Architecture

The frontend consumes the backend through two communication layers: REST for authentication, user search, chat retrieval and message history; Socket.IO for real-time messaging, presence, typing and read receipts.

Architecture: React UI → Router → Pages/Components → API Services → Express REST API

Real-time path: React UI ↔ Socket.IO Client ↔ Socket.IO Server ↔ Connected Users

Authentication: JWT returned by login is used to authenticate protected API requests and the Socket.IO connection. The frontend must never contain or expose the backend JWT secret.

3. Frontend Folder Structure

```
frontend/
├── src/
│   ├── assets/
│   ├── components/
│   ├── auth/
│   ├── chat/
│   ├── common/
│   ├── layout/
│   ├── users/
│   ├── pages/
│   ├── Login.jsx
│   ├── Register.jsx
│   ├── ChatPage.jsx
│   ├── ProfilePage.jsx
│   ├── NotFound.jsx
│   ├── layouts/
│   ├── AppLayout.jsx
│   ├── context/
│   ├── AuthContext.jsx
│   ├── ChatContext.jsx
│   ├── hooks/
│   ├── useAuth.js
│   ├── useChat.js
│   ├── useSocket.js
│   ├── useDebounce.js
│   ├── services/
│   ├── api.js
│   ├── authService.js
│   ├── userService.js
│   ├── chatService.js
│   ├── messageService.js
│   ├── socket/
│   ├── socket.js
│   ├── routes/
│   ├── ProtectedRoute.jsx
│   ├── utils/
│   ├── storage.js
│   ├── formatDate.js
│   ├── messageHelpers.js
│   ├── App.jsx
│   ├── main.jsx
│   ├── .env
│   ├── .env.example
│   ├── package.json
│   └── README.md
```

4. Core Screens and UI

Authentication Screens

- Register: username, email and password fields with validation, loading state and backend error display.
- Login: email/username and password fields according to the backend contract, persistent authenticated session handling, validation and error states.
- Authenticated users are redirected away from authentication pages into the main chat application.

Main Chat Application

- Responsive two-panel desktop layout: conversation/sidebar area and active chat area.
- Mobile layout collapses the sidebar and opens the selected conversation as a dedicated chat view.
- Chat list displays avatar, username, last message, updated time and unread/read state.
- Active conversation header displays avatar, username, online/offline status and last-seen information.
- Message area renders sent and received messages differently, with timestamps and read state.
- Composer supports text entry, send action, disabled/loading state and Enter-to-send behavior without breaking multiline input if implemented.
- Typing indicator appears when the other participant emits typing_started.
- Empty states explain what to do when no conversation is selected or no chats exist.

User Search

- Search users by username using GET /api/users/search?q=. Debounce requests to avoid unnecessary API traffic.
- Show loading, empty-result, error and result states.
- Selecting a result calls POST /api/chats and opens the returned one-to-one conversation.

Profile / Session UI

- Show the authenticated user's username and avatar when available.
- Provide logout and clear client-side authenticated state.
- Do not expose passwordHash or private backend fields.

5. REST API Integration

Method	Endpoint	Frontend responsibility
POST	/api/auth/register	Create account and establish post-registration flow
POST	/api/auth/login	Authenticate user and store session token
GET	/api/auth/me	Restore current user on application startup
GET	/api/users/search?q=	Debounced username search
GET	/api/users/:id	Load public user profile when needed
POST	/api/chats	Create or return existing one-to-one chat
GET	/api/chats	Load authenticated user's conversations
GET	/api/chats/:id	Load selected chat details
DELETE	/api/chats/:id	Archive/delete chat only if backend supports it
GET	/api/messages/:chatId	Load paginated message history
POST	/api/messages	Optional REST message fallback
PATCH	/api/messages/:id/read	Mark a message as read
DELETE	/api/messages/:id	Delete own message if enabled by backend

6. Socket.IO Client Event Design

The client connects to Socket.IO only after authentication and supplies the JWT according to the backend's expected authentication mechanism. Socket lifecycle must handle connect, disconnect, reconnect and authentication errors.

Client → Server:

- `join_chat { chatId }` when a conversation becomes active.
- `leave_chat { chatId }` when leaving or switching conversations.
- `send_message { chatId, text }` when the user sends a message.
- `typing { chatId }` while the user is typing, with throttling/debouncing.
- `stop_typing { chatId }` when typing stops.
- `message_read { messageId, chatId }` after the relevant message is viewed.

Server → Client:

- `message_received { message }` — append/update the message in the active conversation and chat list.
- `message_sent { message }` — reconcile the sent message with the persisted server version.
- `user_online { userId }` — update presence.
- `user_offline { userId, lastSeen }` — update presence and last-seen display.
- `typing_started { userId, chatId }` — show typing indicator.
- `typing_stopped { userId, chatId }` — hide typing indicator.
- `message_read { messageId, userId }` — update read status.
- `error { message }` — show a safe user-facing error.

7. Frontend State Design

Keep server-derived state and UI state organized so real-time updates do not create duplicate messages or stale conversations.

- Auth state: token/session status, current user, authentication loading and auth error.
- User state: search query, search results, search loading and search error.
- Chat state: chat list, activeChatId, chat details, unread counts and chat loading.
- Message state: messages by chat, pagination state, sending state and message errors.
- Presence state: online user IDs, lastSeen values and typing state.
- UI state: mobile sidebar visibility, composer value, selected conversation and transient notifications.

8. Message Lifecycle

- On login, authenticate and initialize the Socket.IO connection.
- Load the user's chat list from `GET /api/chats`.
- When a chat is opened, join its Socket.IO room and fetch paginated history.
- Render historical messages in chronological order.
- When sending, emit `send_message` with `chatId` and `text`; do not trust a client-supplied sender ID.
- When `message_received` arrives, insert the server message exactly once and update the chat preview.
- When `message_sent` arrives, reconcile the server-confirmed message with any temporary UI state if optimistic rendering is used.
- When a message becomes read, emit `message_read` and/or call the read endpoint according to the backend contract.
- When switching chats, leave the previous room and join the new room.

9. Authentication Flow

- Register → validate form → POST /api/auth/register → handle success/error → move to login or authenticated flow according to API response.
- Login → validate credentials → POST /api/auth/login → receive JWT → store token using the chosen client-side storage strategy → load /api/auth/me.
- Application startup → restore token → call /api/auth/me → establish Socket.IO connection only when authenticated.
- Protected routes render only for authenticated users.
- 401 responses clear invalid session state and redirect to login.
- Logout disconnects the socket, clears session state and returns the user to the login page.

10. Responsive UX Requirements

- Desktop: persistent conversation list beside active chat.
- Tablet: flexible sidebar width with readable message area.
- Mobile: one screen at a time; conversation list and active chat transition cleanly.
- Composer remains usable above the mobile keyboard.
- Buttons, inputs and interactive elements have clear hover, focus, active and disabled states.
- Use accessible labels, keyboard navigation, semantic elements and sufficient visual contrast.

11. Loading, Error and Empty States

- Skeleton/loading UI for initial chat list and message history.
- Inline validation for authentication and search forms.
- Friendly API error messages without exposing raw server/database errors.
- Reconnect indicator when Socket.IO disconnects.
- Empty search state when no username matches.
- Empty chat state when the user has no conversations.
- Pagination/loading indicator when older messages are requested.

12. Security and Client-Side Rules

- Never include JWT_SECRET or any backend secret in frontend code.
- Never trust client-provided sender identity.
- Never display passwordHash or other private fields.
- Do not place sensitive data in URLs or logs.
- Escape/render user-generated message text safely; do not inject arbitrary HTML.
- Use the backend as the authority for authorization and message ownership.
- Configure the frontend API and Socket.IO base URLs through environment variables.

13. Environment Variables

```
VITE_API_URL=http://localhost:5000/api VITE_SOCKET_URL=http://localhost:5000
```

14. MVP Development Order

- Phase 1 — Vite + React project setup, Tailwind CSS and routing.

- Phase 2 — Authentication pages, API client and protected routes.
- Phase 3 — Current-user session restoration.
- Phase 4 — Chat list and user search.
- Phase 5 — Chat details and paginated message history.
- Phase 6 — Socket.IO connection and real-time send/receive.
- Phase 7 — Online/offline presence and lastSeen.
- Phase 8 — Typing indicators and read receipts.
- Phase 9 — Responsive polish, loading/error states and accessibility.
- Phase 10 — Production environment configuration and deployment.

15. AI Coding / Frontend Generation Prompt

Use the following prompt with an AI coding tool to generate the frontend against the backend system design.

Build a production-ready MVP frontend for a MERN real-time one-to-one chat application. The backend already exists and exposes the REST API and Socket.IO events defined below. Do not build a backend. Build only the frontend and make the integration contract explicit and configurable.

TECH STACK

- React.js with Vite.
- Tailwind CSS for responsive styling.
- React Router for application routing and protected routes.
- Axios or Fetch for REST API requests.
- Socket.IO Client for real-time communication.
- Context API or Zustand for application state.
- React Hook Form + Zod for form validation.
- Lucide React or an equivalent icon library.

CORE FEATURES

- Registration and login UI.
- JWT session handling and protected routes.
- Restore the authenticated user using GET /api/auth/me.
- Search users by username.
- Start or open a one-to-one conversation.
- Prevent duplicate UI conversations when the backend returns an existing chat.
- Display authenticated user's chat list.
- Display paginated message history.
- Send and receive messages in real time through Socket.IO.
- Show online/offline status and lastSeen.
- Show typing indicators.
- Show message read status.
- Responsive desktop/tablet/mobile chat interface.
- Loading, empty, error, reconnecting and disabled states.

REST CONTRACT

```
POST /api/auth/register POST /api/auth/login GET /api/auth/me GET /api/users/search?q= GET
/api/users/:id POST /api/chats GET /api/chats GET /api/chats/:id DELETE /api/chats/:id GET
/api/messages/:chatId POST /api/messages PATCH /api/messages/:id/read DELETE /api/messages/:id
```

SOCKET.IO CONTRACT

```
Client emits: join_chat { chatId } leave_chat { chatId } send_message { chatId, text } typing { chatId
} stop_typing { chatId } message_read { messageId, chatId } Server emits: message_received { message }
message_sent { message } user_online { userId } user_offline { userId, lastSeen } typing_started {
userId, chatId } typing_stopped { userId, chatId } message_read { messageId, userId } error { message }
```

UI REQUIREMENTS

- Create a clean modern messaging interface with a professional visual hierarchy.
- Use reusable components rather than putting all chat logic into one page.

- Desktop layout should contain a conversation sidebar and active chat panel.
- Mobile layout should switch naturally between conversation list and active conversation.
- Include avatars, usernames, timestamps, online indicators, unread indicators and message read state.
- Use subtle animations only where they improve usability; do not make the interface distracting.
- Provide clear focus states and keyboard accessibility.
- Keep message text safe and render it as text rather than arbitrary HTML.

COMPONENTS TO CREATE

- AuthForm, LoginForm, RegisterForm.
- ProtectedRoute and AppLayout.
- Sidebar, ChatList, ChatListItem.
- UserSearch, SearchResultItem.
- ChatHeader, MessageList, MessageBubble, MessageComposer.
- TypingIndicator, OnlineStatus, EmptyChat, LoadingState, ErrorState.
- ProfileMenu and LogoutButton.

STATE AND INTEGRATION RULES

- Centralize API configuration and automatically attach the JWT to protected requests.
- Create a reusable Socket.IO client/service and clean up listeners on unmount.
- Avoid duplicate Socket.IO listeners during React re-renders.
- Normalize or consistently structure chat/message state so real-time events update the correct conversation.
- When receiving a message for another chat, update that chat's preview/unread state without unexpectedly switching the active conversation.
- When the active chat receives a message, append it immediately and keep the message list scrolled appropriately.
- Handle reconnects without creating duplicate messages or listeners.
- Use the backend response as the source of truth for IDs, timestamps, sender identity and persistence.

ERROR HANDLING

- Handle validation errors, unauthorized sessions, network failures, Socket.IO errors and server errors.
- Display safe, human-readable messages.
- Do not expose stack traces, database errors, JWT secrets or internal implementation details.

DEVELOPER EXPERIENCE

- Provide package.json, .env.example, complete source files and README.
- Document how to start the frontend and configure the backend URL.
- Document the REST and Socket.IO integration points.
- Use clean naming, reusable components, predictable state management and maintainable code.
- Keep the MVP intentionally focused. Do not implement group chats, payments, stories, video calls, complex media uploads or unrelated social features.

EXPECTED FRONTEND RESULT

The completed frontend should support the complete user journey defined by the backend: register → login → restore session → search username → open/create one-to-one chat → load paginated history → join Socket.IO room → send/receive real-time messages → show online/offline status → show typing → mark messages read → logout.

The implementation should be ready to connect to the backend without changing the backend contract.

16. Frontend–Backend Integration Checklist

Area	Frontend must match
Auth	JWT login, /api/auth/me, protected REST requests, authenticated Socket.IO connection
Users	Username search and public profile retrieval
Chats	One-to-one chat creation/retrieval and chat list
Messages	Paginated history, send, receive, read and optional delete
Realtime	All documented Socket.IO events and clean listener lifecycle
Presence	Online/offline and lastSeen
Typing	typing / stop_typing and server typing events
Security	No secrets, no passwordHash, backend remains authorization authority
MVP scope	No unnecessary group/social/payment/video features